HS PF

Programmieren in Python

Python-Grundlagen: Numpy Teil 2

Prof. Dr.-Ing. Stefan Kray

IO mit Numpy

- Numpy kann Textdateien direkt einladen
- Wird als np.array geladen
- `data=np.loadtxt('data.txt')`

numpy.loadtxt(*fname, dtype=<class 'float'>, comments='#', delimiter=None, converters=None, skiprows=0, usecols=None, unpack=False, ndmin=0, encoding='bytes', max_rows=None, *, like=None*) ¶

[source]

Load data from a text file.

Each row in the text file must have the same number of values.

Parameters:

fname : *file, str, or pathlib.Path*

File, filename, or generator to read. If the filename extension is `.gz` or `.bz2`, the file is first decompressed. Note that generators should return byte strings.

dtype : *data-type, optional*

Data-type of the resulting array; default: float. If this is a structured data-type, the resulting array will be 1-dimensional, and each row will be interpreted as an element of the array. In this case, the number of columns used must match the number of fields in the data-type.

comments : *str or sequence of str, optional*

The characters or list of characters used to indicate the start of a comment. None implies no comments. For backwards compatibility, byte strings will be decoded as 'latin1'. The default is '#'.

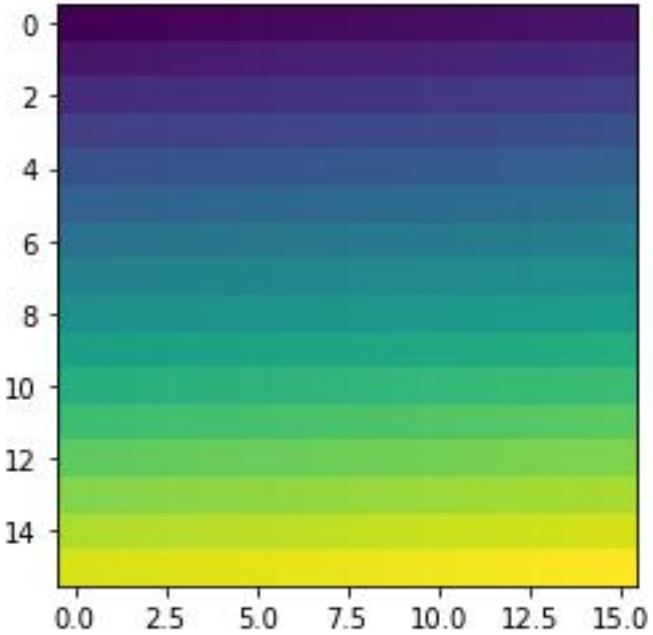
delimiter : *str, optional*

The string used to separate values. For backwards compatibility, byte strings will be decoded as 'latin1'. The default is whitespace.

Arrays umformen mit np.reshape

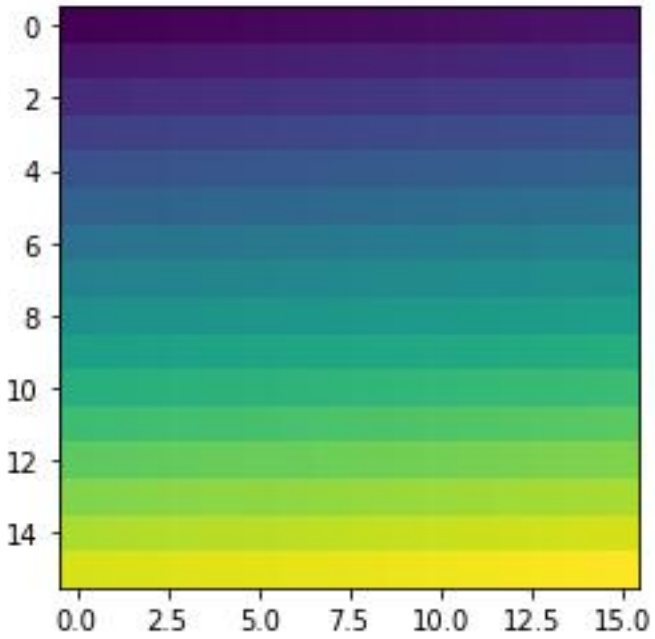
- Häufig liegen Daten als linearer Vektor vor
- Mit weiteren Metainformationen müssen diese Daten umgeformt werden
- In NumPy können Arrays mit dem Befehl „reshape“ auf eine andere Dimensionalität gebracht werden
- Voraussetzung: Größe muss passen
- Beispiel:

```
a=np.linspace(0,256,256) #a=0..255 (256 Werte)
b=a.reshape((16,16)) #b=16x16 2D-Bild
plt.imshow(b)
```



Arrays umformen mit np.flatten

- Umgekehrte Richtung: `np.flatten()`
- `b.flatten()`



m x n (Zeilen x Spalten)





m*n Werte

Neue Dimension hinzufügen

- np.newaxis „schiebt“ Dimension ein:

```
data=np.ones((3,3))
#array([[1., 1., 1.],
#       [1., 1., 1.],
#       [1., 1., 1.]])
data2=data[:, np.newaxis,:]
#array([[[1., 1., 1.]],
#       [[1., 1., 1.]],
#       [[1., 1., 1.]]])
data2[:,0,:]
#array([[1., 1., 1.],
#       [1., 1., 1.],
#       [1., 1., 1.]])
```

Aufgabe dataimg.py

- Schreiben Sie ein Programm, dass die Datei „dataimg.txt“ lädt
- Erzeugen Sie einen Plot der geladenen daten
- Formen Sie die Daten in ein 2D Array um mit der Größe 256x256 Pixel
- Stellen Sie dieses Array als Bild dar

NumPy Indexing (Single Element Indexing)

- Auf NumPy-Arrays wird wie auf Listen zugegriffen

```
x=np.array([3,6,9,12,15,18,21,24,27,30])
print(x[4]) #15
print(x[-1]) #30

x=np.array([[1,2,3],[4,5,6]])
print(x[1][2])
print(x[1,2])
```

NumPy Slicing

- Nicht nur einzelne Indizes möglich
- Bereiche wie bei Listen erlaubt
- Auch mehrdimensional

```
#data=np.array([[1, 2, 3],
#                [4, 5, 6],
#                [7, 8, 9]])

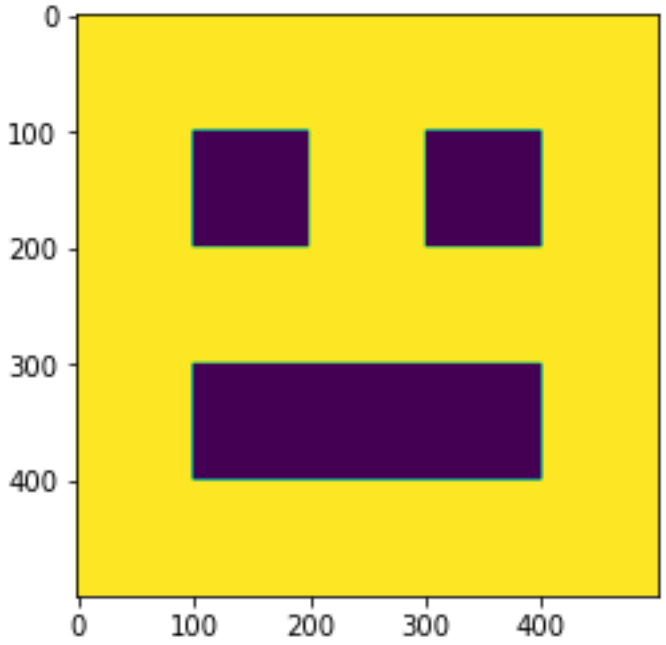
data[:,0] #[1,4,7]
data[1,:] #[4,5,6]
data[::2,1] #[2,8]

data[1:3,1:3]
#array([[5, 6],
#       [8, 9]])

data[0:3:2,0:3:2] oder data[::2,::2]
#array([[1, 3],
#       [7, 9]])
```

Aufgabe smiley.py

- Erzeugen Sie ein 2D-Numpy-Array der Größe 500x500
  - Füllen Sie es mit Einsen
  - Schneiden Sie Löcher in Form eines Smileys wie folgt (s.u.)
  - Stellen Sie das Ergebnis mit imshow dar



NumPy Indexing (2)

- Auch Indizierung über mehrere Indizes möglich:

```
data = np.array([[1, 2], [3, 4], [5, 6]])
#array([[1, 2],
#       [3, 4],
#       [5, 6]])
data[2,1] #6
data[[2],[1]] #[6]
data[0,1] #2

y_ind=[2,1]
x_ind=[0,1]
data[[2,1],[0,1]] # [5,4]
data[y_ind,x_ind] # [5,4] ; Achtung: advanced indexing als Zeilen und Spalten getrennt
```

NumPy Indexing (3)

- Auch Indizierung über mehrere Indizes möglich:

```
data = np.array([[1, 2], [3, 4], [5, 6]])
#array([[1, 2],
#       [3, 4],
#       [5, 6]])
data[2,1] #6
data[[2],[1]] #[6]
data[0,1] #2

y_ind=[2,1]
x_ind=[0,1]
data[y_ind,x_ind] #[5,4] ; Achtung: advanced indexing als Zeilen und Spalten getrennt

#Per Punktpaare:
addr=np.array([[2,1],[0,1]])
data[addr[:,0],addr[:,1]] #Form per slicing umwandeln -> [6,2]
```

NumPy Indexing (4)

- Indizierung über Boolean-Arrays:

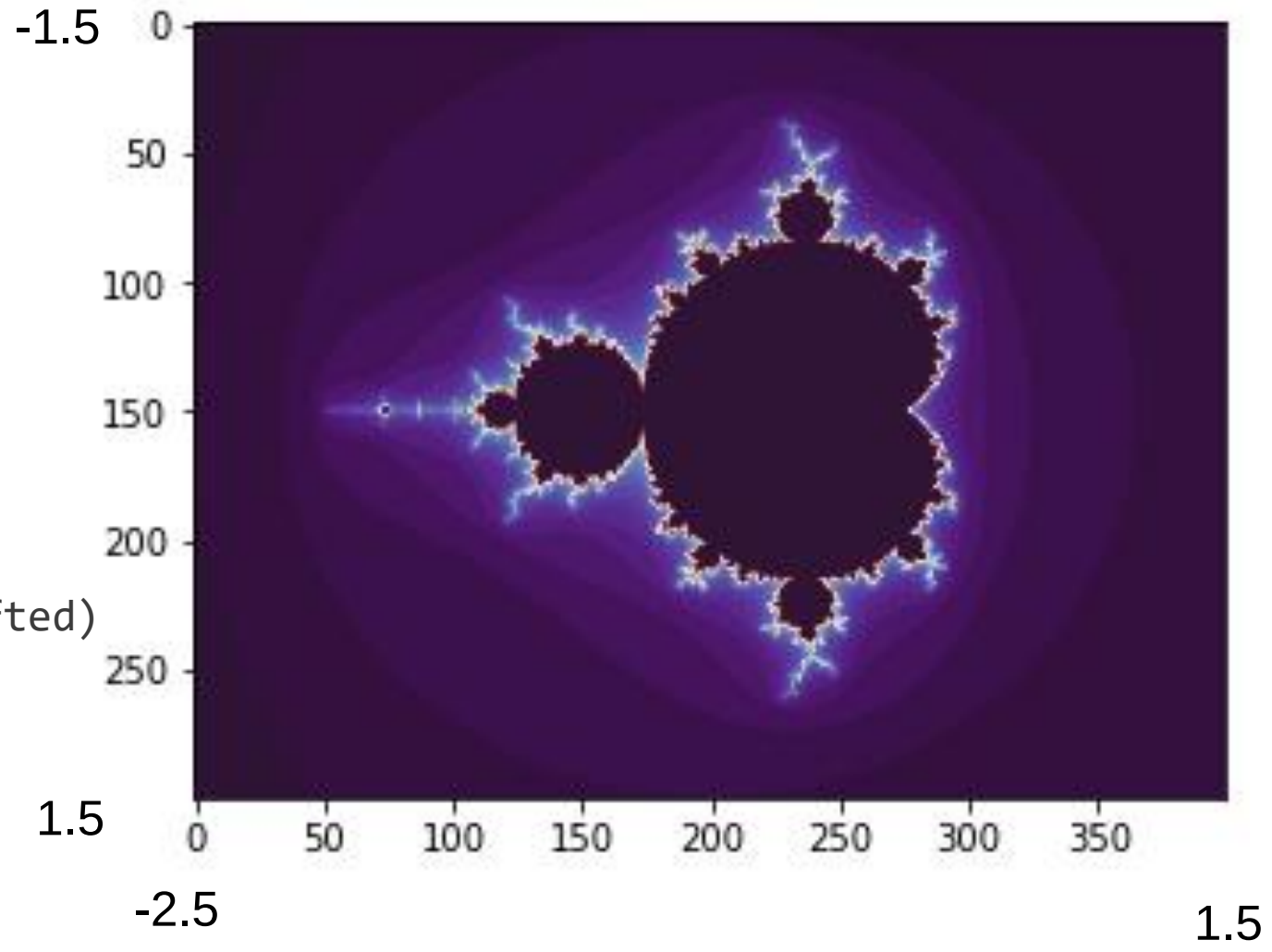
```
data = np.array([[1, 2], [3, 4], [5, 6]])
#array([[1, 2],
#       [3, 4],
#       [5, 6]])

xy_ind=np.zeros((3,2)).astype(np.bool)
xy_ind[2,0]=True
xy_ind[1,1]=True
#array([[False, False],
#       [False,  True],
#       [ True, False]])
data[xy_ind] #[4,5]
```

Mandelbrot / Apfelmännchen

```
import numpy as np
import matplotlib.pyplot as plt

yy, xx = np.mgrid[-1.5:1.5:300j,
-2.5:1.5:400j]
C=(xx+yy*1j)
Z = C*0
T = np.zeros(C.shape)
for k in range(50):
    M = abs(Z) < 2.5
    Z[M] = Z[M] ** 2 + C[M]
    T[M] = k + 1
plt.imshow(T, cmap=plt.cm.twilight_shifted)
```



Mandelbrot in C++ (ohne Grafikausgabe, Spezialprogramm zur Anzeige)

```
#include <iostream>
#include <fstream>
#include <complex>

#define WIDTH 600
#define HEIGHT 600

using namespace std;

int value (int x, int y)  {
   complex<float> point((float)x/WIDTH-1.5, (float)y/HEIGHT-0.5);
   complex<float> z(0, 0);
   int nb_iter = 0;
   while (abs (z) < 2 && nb_iter <= 20) {
      z = z * z + point;
      nb_iter++;
   }
   if (nb_iter < 20)
     return (255*nb_iter)/20;
   else
     return 0;
}
```

```
int main ()  {

   ofstream my_Image ("mandelbrot.ppm");



   if (my_Image.is_open ()) {
      my_Image << "P3\n" << WIDTH << " " << HEIGHT << " 255\n";
      for (int i = 0; i < WIDTH; i++) {
         for (int j = 0; j < HEIGHT; j++)  {
            int val = value(i, j);
            my_Image << val << ' ' << 0 << ' ' << 0 << "\n";
         }
      }
      my_Image.close();
   }
   else
    cout << "Could not open the file";


   return 0;
}
```

Quelle: https://github.com/SlachPi/mandelbrot_cpp/blob/master/mandelbrot.cpp

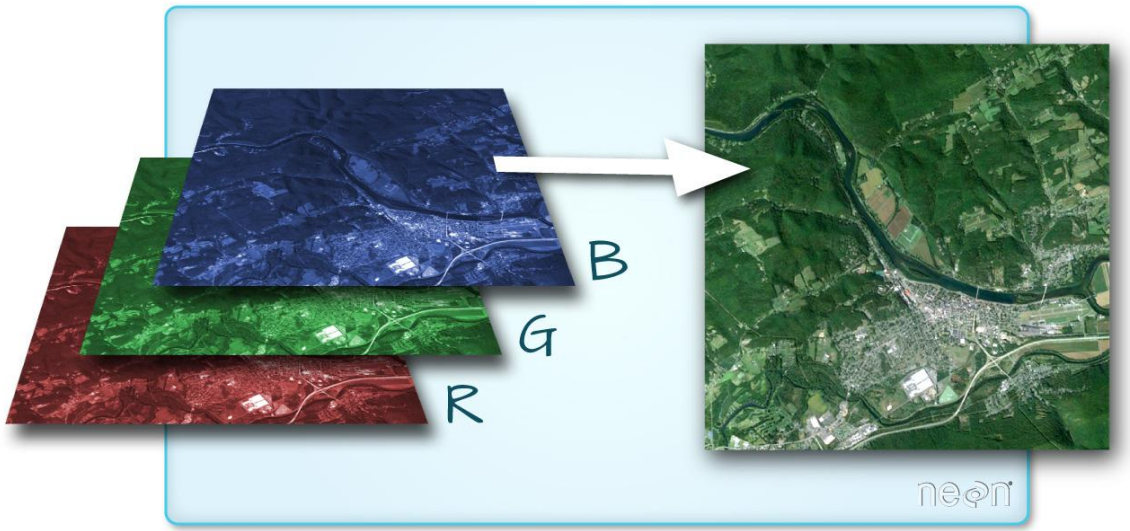
NumPy Operationen in mehreren Dimensionen

- NumPy-Befehle lassen sich in „Richtungen“ anwenden
- Parameter „axis“ steuert die Richtung, in der die Operation angewandt wird
- möglich bei den Befehlen: np.mean, np.sum, np.diff, np.max, np.min und einigen weiteren
- Beispiel:

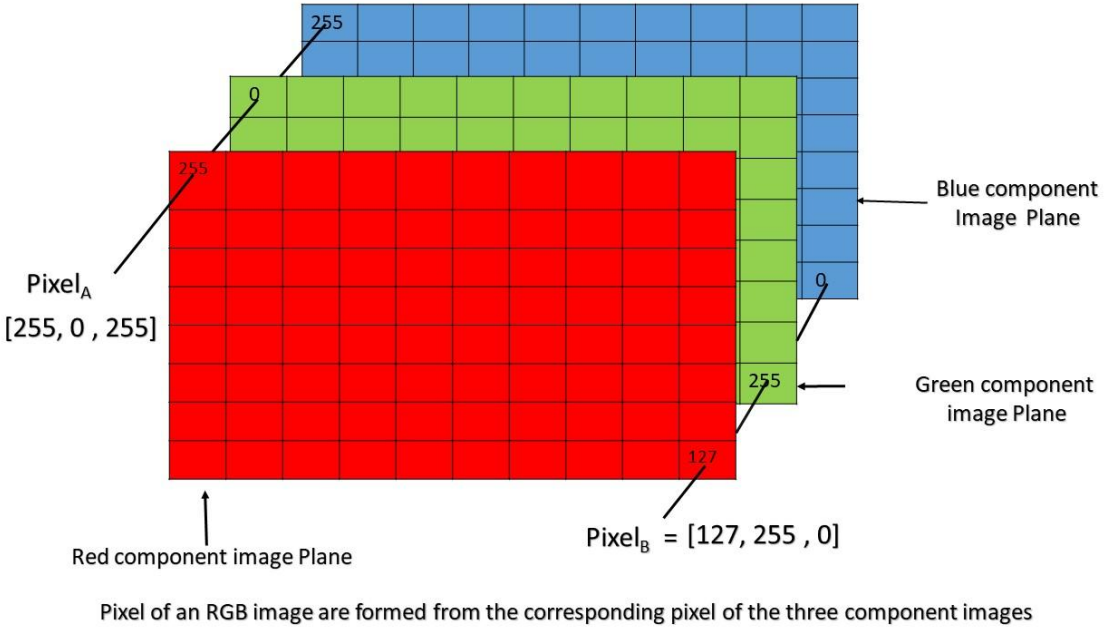
```
data=np.array([[1, 2, 3],
               [4, 5, 6],
               [7, 8, 9]])
np.sum(data,axis=0) #[12,15,18]
np.sum(data,axis=1) #[6,15,24]
data.sum(axis=0) #[12,15,18]
data.sum(0) #[12,15,18]
data.max(axis=0) #[7,8,9]
data.max(axis=1) #[3,6,9]
data.mean(0) #[4.0,5.0,6.0]
data.mean(1) #[2.0,5.0,8.0]
data.mean(axis=0).mean(axis=0) #5.0
data.mean() #5.0
```

Einschub: Farbbilder

- Aufbau von Farbbildern:



https://www.neonscience.org/resources/learning-hub/tutorials/dc-multiband-rasters-r



Pixel of an RGB image are formed from the corresponding pixel of the three component images

https://www.geeksforgeeks.org/matlab-rgb-image-representation/

Aufgabe numpy_slicing.py

1. Laden Sie das Bild „testbild.png“ mit der Matplotlib:

```
import matplotlib.image as mpimg
img = mpimg.imread('testbild.png').copy() #Format (1920,2560,3)
```

1. Stellen Sie das Bild „img“ mit der Matplotlib dar
2. Transformieren (skalieren) Sie die geladenen Daten in ein Farbbild mit 960 Zeilen und 1280 Spalten. Speichern Sie den transformierten Datensatz in der Variable „img2“.
3. Erzeugen Sie aus dem Bild mit drei Farbkanälen ein neues Bild „img3“ (960x1280), dass den Mittelwert aller Farben beinhaltet.
4. Plotten Sie die 500te Zeile aus dem Bild „img3“.

5. Zusatzaufgabe: Ändern Sie alle hellen weißen Pixel (mittlere Farbe>=0.99) von img3 in schwarz